JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Double

java.lang.Object
    java.lang.Number
        java.lang.Double

All Implemented Interfaces:

`Serializable`, `Comparable<Double>`, `Constable`, `ConstantDesc`

```
public final class Double
extends Number
implements Comparable<Double>, Constable, ConstantDesc
```

The `Double` class is the wrapper class for values of the primitive type `double`. An object of type `Double` contains a single field whose type is `double`.

In addition, this class provides several methods for converting a `double` to a `String` and a `String` to a `double`, as well as other constants and methods useful when dealing with a `double`.

This is a value-based class; programmers should treat instances that are equal as interchangeable and should not use instances for synchronization, or unpredictable behavior may occur. For example, in a future release, synchronization may fail.

Floating-point Equality, Equivalence, and Comparison

IEEE 754 floating-point values include finite nonzero values, signed zeros (`+0.0` and `-0.0`), signed infinities (positive infinity and negative infinity), and NaN (not-a-number).

An *equivalence relation* on a set of values is a boolean relation on pairs of values that is reflexive, symmetric, and transitive. For more discussion of equivalence relations and object equality, see the `Object.equals` specification. An equivalence relation partitions the values it operates over into sets called *equivalence classes*. All the members of the equivalence class are equal to each other under the relation. An equivalence class may contain only a single member. At least for some purposes, all the members of an equivalence class are substitutable for each other. In particular, in a numeric expression equivalent values can be *substituted* for one another without changing the result of the expression, meaning changing the equivalence class of the result of the expression.

Notably, the built-in == operation on floating-point values is *not* an equivalence relation. Despite not defining an equivalence relation, the semantics of the IEEE 754 == operator were deliberately designed to meet other needs of

numerical computation. There are two exceptions where the properties of an equivalence relation are not satisfied by == on floating-point values:

- If `v1` and `v2` are both NaN, then `v1 == v2` has the value `false`. Therefore, for two NaN arguments the *reflexive* property of an equivalence relation is *not* satisfied by the == operator.
- If `v1` represents `+0.0` while `v2` represents `-0.0`, or vice versa, then `v1 == v2` has the value `true` even though `+0.0` and `-0.0` are distinguishable under various floating-point operations. For example, `1.0/+0.0` evaluates to positive infinity while `1.0/-0.0` evaluates to *negative* infinity and positive infinity and negative infinity are neither equal to each other nor equivalent to each other. Thus, while a signed zero input most commonly determines the sign of a zero result, because of dividing by zero, `+0.0` and `-0.0` may not be substituted for each other in general. The sign of a zero input also has a non-substitutable effect on the result of some math library methods.

For ordered comparisons using the built-in comparison operators (<, <=, etc.), NaN values have another anomalous situation: a NaN is neither less than, nor greater than, nor equal to any value, including itself. This means the *trichotomy of comparison* does *not* hold.

To provide the appropriate semantics for `equals` and `compareTo` methods, those methods cannot simply be wrappers around == or ordered comparison operations. Instead, `equals` uses representation equivalence, defining NaN arguments to be equal to each other, restoring reflexivity, and defining `+0.0` to *not* be equal to `-0.0`. For comparisons, `compareTo` defines a total order where `-0.0` is less than `+0.0` and where a NaN is equal to itself and considered greater than positive infinity.

The operational semantics of `equals` and `compareTo` are expressed in terms of bit-wise converting the floating-point values to integral values.

The *natural ordering* implemented by `compareTo` is consistent with equals. That is, two objects are reported as equal by `equals` if and only if `compareTo` on those objects returns zero.

The adjusted behaviors defined for `equals` and `compareTo` allow instances of wrapper classes to work properly with conventional data structures. For example, defining NaN values to be `equals` to one another allows NaN to be used as an element of a `HashSet` or as the key of a `HashMap`. Similarly, defining `compareTo` as a total ordering, including `+0.0`, `-0.0`, and NaN, allows instances of wrapper classes to be used as elements of a `SortedSet` or as keys of a `SortedMap`.

Comparing numerical equality to various useful equivalence relations that can be defined over floating-point values:

numerical equality (== operator): (*Not* an equivalence relation)

> Two floating-point values represent the same extended real number. The extended real numbers are the real numbers augmented with positive infinity and negative infinity. Under numerical equality, `+0.0` and `-0.0` are equal since they both map to the same real value, 0. A NaN does not map to any real number and is not equal to any value, including itself.

bit-wise equivalence:

> The bits of the two floating-point values are the same. This equivalence relation for `double` values `a` and `b` is implemented by the expression `Double.doubleTo**Raw**LongBits(a) == Double.doubleTo**Raw**LongBits(b)` Under this relation, `+0.0` and `-0.0` are distinguished from each other and every bit pattern encoding a NaN is distinguished from every other bit pattern encoding a NaN.

representation equivalence:

> The two floating-point values represent the same IEEE 754 *datum*. In particular, for finite values, the sign, exponent, and significand components
> of the floating-point values are the same. Under this relation:
>
> - `+0.0` and `-0.0` are distinguished from each other.
> - every bit pattern encoding a NaN is considered equivalent to each other
> - positive infinity is equivalent to positive infinity; negative infinity is equivalent to negative infinity.
>
> Expressions implementing this equivalence relation include:
>
> - `Double.doubleToLongBits(a) == Double.doubleToLongBits(b)`
> - `Double.valueOf(a).equals(Double.valueOf(b))`
> - `Double.compare(a, b) == 0`
>
> Note that representation equivalence is often an appropriate notion of equivalence to test the behavior of math libraries.

For two binary floating-point values `a` and `b`, if neither of `a` and `b` is zero or NaN, then the three relations numerical equality, bit-wise equivalence, and representation equivalence of `a` and `b` have the same `true`/`false` value. In other words, for binary floating-point values, the three relations only differ if at least one argument is zero or NaN.

Decimal ↔ Binary Conversion Issues

Many surprising results of binary floating-point arithmetic trace back to aspects of decimal to binary conversion and binary to decimal conversion. While integer values can be exactly represented in any base, which fractional values can be exactly represented in a base is a function of the base. For example, in base 10, 1/3 is a repeating fraction (0.33333....); but in base 3, 1/3 is exactly $0.1_{(3)}$, that is 1×3^{-1}. Similarly, in base 10, 1/10 is exactly

representable as 0.1 (1×10^{-1}), but in base 2, it is a repeating fraction ($0.0001100110011\ldots_{(2)}$).

Values of the `float` type have 24 bits of precision and values of the `double` type have 53 bits of precision. Therefore, since 0.1 is a repeating fraction in base 2 with a four-bit repeat, `0.1f != 0.1d`. In more detail, including hexadecimal floating-point literals:

- The exact numerical value of `0.1f` (`0x1.99999a0000000p-4f`) is 0.100000001490116119384765625.
- The exact numerical value of `0.1d` (`0x1.999999999999ap-4d`) is 0.1000000000000000055511151231257827021181583404541015625.

These are the closest `float` and `double` values, respectively, to the numerical value of 0.1. These results are consistent with a `float` value having the equivalent of 6 to 9 digits of decimal precision and a `double` value having the equivalent of 15 to 17 digits of decimal precision. (The equivalent precision varies according to the different relative densities of binary and decimal values at different points along the real number line.)

This representation hazard of decimal fractions is one reason to use caution when storing monetary values as `float` or `double`. Alternatives include:

- using `BigDecimal` to store decimal fractional values exactly
- scaling up so the monetary value is an integer — for example, multiplying by 100 if the value is denominated in cents or multiplying by 1000 if the value is denominated in mills — and then storing that scaled value in an integer type

For each finite floating-point value and a given floating-point type, there is a contiguous region of the real number line which maps to that value. Under the default round to nearest rounding policy (JLS §15.4), this contiguous region for a value is typically one ulp (unit in the last place) wide and centered around the exactly representable value. (At exponent boundaries, the region is asymmetrical and larger on the side with the larger exponent.) For example, for `0.1f`, the region can be computed as follows:
// Numeric values listed are exact values
oneTenthApproxAsFloat = 0.100000001490116119384765625;
ulpOfoneTenthApproxAsFloat = Math.ulp(0.1f) = 7.450580596923828125E-9;
// Numeric range that is converted to the float closest to 0.1, _excludes_ endpoints
(oneTenthApproxAsFloat - ½ulpOfoneTenthApproxAsFloat,
oneTenthApproxAsFloat + ½ulpOfoneTenthApproxAsFloat) =
(0.099999997764825820922851562 5, 0.100000005215406417846679687 5)

In particular, a correctly rounded decimal to binary conversion of any string representing a number in this range, say by `Float.parseFloat(String)`, will be converted to the same value:

```
Float.parseFloat("0.099999997764825820922851562500000001"); // rounds up to oneTent
Float.parseFloat("0.099999998");                            // rounds up to oneTent
Float.parseFloat("0.1");                                    // rounds up to oneTent
Float.parseFloat("0.100000001490116119384765625");          // exact conversion
Float.parseFloat("0.100000005215406417846679687");          // rounds down to oneTe
Float.parseFloat("0.100000005215406417846679687499999");    // rounds down to oneTe
```

Similarly, an analogous range can be constructed for the `double` type based on the exact value of `double` approximation to `0.1d` and the numerical value of `Math.ulp(0.1d)` and likewise for other particular numerical values in the `float` and `double` types.

As seen in the above conversions, compared to the exact numerical value the operation would have without rounding, the same floating-point value as a result can be:

- greater than the exact result
- equal to the exact result
- less than the exact result

A floating-point value doesn't "know" whether it was the result of rounding up, or rounding down, or an exact operation; it contains no history of how it was computed. Consequently, the sum of

```
0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f + 0.1f;
// Numerical value of computed sum: 1.000000119209289550781 25,
// the next floating-point value larger than 1.0f, equal to Math.nextUp(1.0f).
```

or

```
0.1d + 0.1d + 0.1d + 0.1d + 0.1d + 0.1d + 0.1d + 0.1d + 0.1d + 0.1d;
// Numerical value of computed sum: 0.999999999999999888977697537484345957636833 1
// the next floating-point value smaller than 1.0d, equal to Math.nextDown(1.0d).
```

should *not* be expected to be exactly equal to 1.0, but only to be close to 1.0. Consequently, the following code is an infinite loop:

```
double d = 0.0;
while (d != 1.0) { // Surprising infinite loop
```

```
  d += 0.1; // Sum never _exactly_ equals 1.0
}
```

Instead, use an integer loop count for counted loops:

```
double d = 0.0;
for (int i = 0; i < 10; i++) {
  d += 0.1;
} // Value of d is equal to Math.nextDown(1.0).
```

or test against a floating-point limit using ordered comparisons (<, <=, >, >=):

```
double d = 0.0;
while (d <= 1.0) {
  d += 0.1;
} // Value of d approximately 1.0999999999999999
```

While floating-point arithmetic may have surprising results, IEEE 754 floating-point arithmetic follows a principled design and its behavior is predictable on the Java platform.

See *Java Language Specification*:

4.2.3 Floating-Point Types and Values
4.2.4 Floating-Point Operations
15.21.1 Numerical Equality Operators == and !=
15.20.1 Numerical Comparison Operators <, <=, >, and >=

Since:

1.0

External Specifications

IEEE Standard for Floating-Point Arithmetic

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**BYTES**	The number of bytes used to represent a `double` value, 8.

static final int	**MAX_EXPONENT**	Maximum exponent a finite `double` variable may have, 1023.
static final double	**MAX_VALUE**	A constant holding the largest positive finite value of type `double`, $(2-2^{-52})\cdot 2^{1023}$.
static final int	**MIN_EXPONENT**	Minimum exponent a normalized `double` variable may have, -1022.
static final double	**MIN_NORMAL**	A constant holding the smallest positive normal value of type `double`, 2^{-1022}.
static final double	**MIN_VALUE**	A constant holding the smallest positive nonzero value of type `double`, 2^{-1074}.
static final double	**NaN**	A constant holding a Not-a-Number (NaN) value of type `double`.
static final double	**NEGATIVE_INFINITY**	A constant holding the negative infinity of type `double`.
static final double	**POSITIVE_INFINITY**	A constant holding the positive infinity of type `double`.
static final int	**PRECISION**	The number of bits in the significand of a `double` value, 53.
static final int	**SIZE**	The number of bits used to represent a `double` value, 64.
static final Class<Double>	**TYPE**	

	The `Class` instance representing the primitive type `double`.

Constructor Summary

Constructors

Constructor	Description
`Double(double value)`	**Deprecated.** It is rarely appropriate to use this constructor.
`Double(String s)`	**Deprecated.** It is rarely appropriate to use this constructor.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
`byte`	`byteValue()`	Returns the value of this `Double` as a `byte` after a narrowing primitive conversion.
`static int`	`compare(double d1, double d2)`	Compares the two specified `double` values.
`int`	`compareTo (Double anotherDouble)`	Compares two `Double` objects numerically.
`Optional<Double>`	`describeConstable()`	Returns an `Optional` containing the nominal descriptor for this instance, which is the instance itself.
`static long`	`doubleToLongBits (double value)`	Returns a representation of the specified floating-point value according to the IEEE 754 floating-

		point "double format" bit layout.
static long	**doubleToRawLongBits**(double value)	Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "double format" bit layout, preserving Not-a-Number (NaN) values.
double	**doubleValue**()	Returns the double value of this Double object.
boolean	**equals**(Object obj)	Compares this object against the specified object.
float	**floatValue**()	Returns the value of this Double as a float after a narrowing primitive conversion.
int	**hashCode**()	Returns a hash code for this Double object.
static int	**hashCode**(double value)	Returns a hash code for a double value; compatible with Double.hashCode().
int	**intValue**()	Returns the value of this Double as an int after a narrowing primitive conversion.
static boolean	**isFinite**(double d)	Returns true if the argument is a finite floating-point value; returns false otherwise (for NaN and infinity arguments).

boolean	**isInfinite**()	Returns true if this Double value is infinitely large in magnitude, false otherwise.
static boolean	**isInfinite**(double v)	Returns true if the specified number is infinitely large in magnitude, false otherwise.
boolean	**isNaN**()	Returns true if this Double value is a Not-a-Number (NaN), false otherwise.
static boolean	**isNaN**(double v)	Returns true if the specified number is a Not-a-Number (NaN) value, false otherwise.
static double	**longBitsToDouble**(long bits)	Returns the double value corresponding to a given bit representation.
long	**longValue**()	Returns the value of this Double as a long after a narrowing primitive conversion.
static double	**max**(double a, double b)	Returns the greater of two double values as if by calling Math.max.
static double	**min**(double a, double b)	Returns the smaller of two double values as if by calling Math.min.
static double	**parseDouble**(String s)	Returns a new double initialized to the value represented by the specified String, as performed by the valueOf method of class Double.

Double	**resolveConstantDesc**(MethodHandles.Lookup lo	Resolves this instance as a ConstantDesc, the result of which is the instance itself.
short	**shortValue**()	Returns the value of this Double as a short after a narrowing primitive conversion.
static double	**sum**(double a, double b)	Adds two double values together as per the + operator.
static String	**toHexString**(double d)	Returns a hexadecimal string representation of the double argument.
String	**toString**()	Returns a string representation of this Double object.
static String	**toString**(double d)	Returns a string representation of the double argument.
static Double	**valueOf**(double d)	Returns a Double instance representing the specified double value.
static Double	**valueOf**(String s)	Returns a Double object holding the double value represented by the argument string s.

Methods declared in class Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Details

POSITIVE_INFINITY

```
public static final double POSITIVE_INFINITY
```

A constant holding the positive infinity of type `double`. It is equal to the value returned by `Double.longBitsToDouble(0x7ff0000000000000L)`.

See Also:

Constant Field Values

NEGATIVE_INFINITY

```
public static final double NEGATIVE_INFINITY
```

A constant holding the negative infinity of type `double`. It is equal to the value returned by `Double.longBitsToDouble(0xfff0000000000000L)`.

See Also:

Constant Field Values

NaN

```
public static final double NaN
```

A constant holding a Not-a-Number (NaN) value of type `double`. It is equivalent to the value returned by `Double.longBitsToDouble(0x7ff8000000000000L)`.

See Also:

Constant Field Values

MAX_VALUE

```
public static final double MAX_VALUE
```

A constant holding the largest positive finite value of type `double`, $(2-2^{-52})\cdot 2^{1023}$. It is equal to the hexadecimal floating-point literal `0x1.fffffffffffffP+1023` and also equal to `Double.longBitsToDouble(0x7fefffffffffffffL)`.

See Also:

Constant Field Values

MIN_NORMAL

```
public static final double MIN_NORMAL
```

A constant holding the smallest positive normal value of type `double`, 2^{-1022}. It is equal to the hexadecimal floating-point literal `0x1.0p-1022` and also equal to `Double.longBitsToDouble(0x0010000000000000L)`.

Since:

1.6

See Also:

Constant Field Values

MIN_VALUE

`public static final double MIN_VALUE`

A constant holding the smallest positive nonzero value of type `double`, 2^{-1074}. It is equal to the hexadecimal floating-point literal `0x0.0000000000001P-1022` and also equal to `Double.longBitsToDouble(0x1L)`.

See Also:

Constant Field Values

SIZE

`public static final int SIZE`

The number of bits used to represent a `double` value, 64.

Since:

1.5

See Also:

Constant Field Values

PRECISION

`public static final int PRECISION`

The number of bits in the significand of a `double` value, 53. This is the parameter N in section §4.2.3 of *The Java Language Specification*.

Since:

19

See Also:

Constant Field Values

MAX_EXPONENT

`public static final int MAX_EXPONENT`

Maximum exponent a finite `double` variable may have, 1023. It is equal to the value returned by `Math.getExponent(Double.MAX_VALUE)`.

Since:

1.6

See Also:

Constant Field Values

MIN_EXPONENT

```
public static final int MIN_EXPONENT
```

Minimum exponent a normalized `double` variable may have, -1022. It is equal to the value returned by `Math.getExponent(Double.MIN_NORMAL)`.

Since:

1.6

See Also:

Constant Field Values

BYTES

```
public static final int BYTES
```

The number of bytes used to represent a `double` value, 8.

Since:

1.8

See Also:

Constant Field Values

TYPE

```
public static final Class<Double> TYPE
```

The `Class` instance representing the primitive type `double`.

Since:

1.1

Constructor Details

Double

```
@Deprecated(since="9")
public Double(double value)
```

Deprecated.
It is rarely appropriate to use this constructor. The static factory `valueOf(double)` is generally a better choice, as it is likely to yield significantly better space and time performance.

Constructs a newly allocated `Double` object that represents the primitive `double` argument.

Parameters:

`value` - the value to be represented by the `Double`.

Double

```
@Deprecated(since="9")
public Double(String s)
       throws NumberFormatException
```

Deprecated.
It is rarely appropriate to use this constructor. Use `parseDouble(String)` to convert a string to a `double` primitive, or use `valueOf(String)` to convert a string to a `Double` object.

Constructs a newly allocated `Double` object that represents the floating-point value of type `double` represented by the string. The string is converted to a `double` value as if by the `valueOf` method.

Parameters:

`s` - a string to be converted to a `Double`.

Throws:

`NumberFormatException` - if the string does not contain a parsable number.

Method Details

toString

```
public static String toString(double d)
```

Returns a string representation of the `double` argument. All characters mentioned below are ASCII characters.

- If the argument is NaN, the result is the string `"NaN"`.
- Otherwise, the result is a string that represents the sign and magnitude (absolute value) of the argument. If the sign is negative, the first character of the result is `'-'` (`'\u002D'`); if the sign is positive, no sign character appears in the result. As for the magnitude *m*:
  - If *m* is infinity, it is represented by the characters `"Infinity"`; thus, positive infinity produces the result `"Infinity"` and negative infinity produces the result `"-Infinity"`.
  - If *m* is zero, it is represented by the characters `"0.0"`; thus, negative zero produces the result `"-0.0"` and positive zero produces the result `"0.0"`.

- Otherwise m is positive and finite. It is converted to a string in two stages:
  - *Selection of a decimal*: A well-defined decimal d_m is selected to represent m. This decimal is (almost always) the *shortest* one that rounds to m according to the round to nearest rounding policy of IEEE 754 floating-point arithmetic.
  - *Formatting as a string*: The decimal d_m is formatted as a string, either in plain or in computerized scientific notation, depending on its value.

A *decimal* is a number of the form $s \times 10^i$ for some (unique) integers $s > 0$ and i such that s is not a multiple of 10. These integers are the *significand* and the *exponent*, respectively, of the decimal. The *length* of the decimal is the (unique) positive integer n meeting $10^{n-1} \le s < 10^n$.

The decimal d_m for a finite positive m is defined as follows:

- Let R be the set of all decimals that round to m according to the usual *round to nearest* rounding policy of IEEE 754 floating-point arithmetic.
- Let p be the minimal length over all decimals in R.
- When $p \ge 2$, let T be the set of all decimals in R with length p. Otherwise, let T be the set of all decimals in R with length 1 or 2.
- Define d_m as the decimal in T that is closest to m. Or if there are two such decimals in T, select the one with the even significand.

The (uniquely) selected decimal d_m is then formatted. Let s, i and n be the significand, exponent and length of d_m, respectively. Further, let $e = n + i - 1$ and let $s_1 \ldots s_n$ be the usual decimal expansion of s. Note that $s_1 \ne 0$ and $s_n \ne 0$. Below, the decimal point `'.'` is `'\u002E'` and the exponent indicator `'E'` is `'\u0045'`.

- Case $-3 \le e < 0$: d_m is formatted as `0.0`$\ldots$`0`$s_1 \ldots s_n$, where there are exactly $-(n + i)$ zeroes between the decimal point and s_1. For example, 123×10^{-4} is formatted as `0.0123`.
- Case $0 \le e < 7$:
  - Subcase $i \ge 0$: d_m is formatted as $s_1 \ldots s_n$`0`$\ldots$`0.0`, where there are exactly i zeroes between s_n and the decimal point. For example, 123×10^2 is formatted as `12300.0`.
  - Subcase $i < 0$: d_m is formatted as $s_1 \ldots s_{n+i} . s_{n+i+1} \cdots s_n$, where there are exactly $-i$ digits to the right of the decimal point. For example, 123×10^{-1} is formatted as `12.3`.

- Case e < -3 or e ≥ 7: computerized scientific notation is used to format d_m. Here e is formatted as by `Integer.toString(int)`.
  - Subcase n = 1: d_m is formatted as s_1`.0E`e. For example, 1 × 10^{23} is formatted as `1.0E23`.
  - Subcase n > 1: d_m is formatted as s_1`.`$s_2 \ldots s_n$`E`e. For example, 123 × 10^{-21} is formatted as `1.23E-19`.

To create localized string representations of a floating-point value, use subclasses of `NumberFormat`.

API Note:

This method corresponds to the general functionality of the convertToDecimalCharacter operation defined in IEEE 754; however, that operation is defined in terms of specifying the number of significand digits used in the conversion. Code to do such a conversion in the Java platform includes converting the `double` to a `BigDecimal` exactly and then rounding the `BigDecimal` to the desired number of digits; sample code:

```
double d = 0.1;
int digits = 25;
BigDecimal bd = new BigDecimal(d);
String result = bd.round(new MathContext(digits,  RoundingMode.HALF_UP));
// 0.1000000000000000055511151
```

Parameters:

`d` - the `double` to be converted.

Returns:

a string representation of the argument.

toHexString

```
public static String toHexString(double d)
```

Returns a hexadecimal string representation of the `double` argument. All characters mentioned below are ASCII characters.

- If the argument is NaN, the result is the string `"NaN"`.
- Otherwise, the result is a string that represents the sign and magnitude of the argument. If the sign is negative, the first character of the result is `'-'` (`'\u002D'`); if the sign is positive, no sign character appears in the result. As for the magnitude m:
  - If m is infinity, it is represented by the string `"Infinity"`; thus, positive infinity produces the result `"Infinity"` and negative infinity produces the result `"-Infinity"`.

- If *m* is zero, it is represented by the string `"0x0.0p0"`; thus, negative zero produces the result `"-0x0.0p0"` and positive zero produces the result `"0x0.0p0"`.
- If *m* is a `double` value with a normalized representation, substrings are used to represent the significand and exponent fields. The significand is represented by the characters `"0x1."` followed by a lowercase hexadecimal representation of the rest of the significand as a fraction. Trailing zeros in the hexadecimal representation are removed unless all the digits are zero, in which case a single zero is used. Next, the exponent is represented by `"p"` followed by a decimal string of the unbiased exponent as if produced by a call to `Integer.toString` on the exponent value.
- If *m* is a `double` value with a subnormal representation, the significand is represented by the characters `"0x0."` followed by a hexadecimal representation of the rest of the significand as a fraction. Trailing zeros in the hexadecimal representation are removed. Next, the exponent is represented by `"p-1022"`. Note that there
must be at least one nonzero digit in a subnormal significand.

Examples

Floating-point Value	Hexadecimal String
1.0	0x1.0p0
-1.0	-0x1.0p0
2.0	0x1.0p1
3.0	0x1.8p1
0.5	0x1.0p-1
0.25	0x1.0p-2
Double.MAX_VALUE	0x1.fffffffffffffp1023
Minimum Normal Value	0x1.0p-1022
Maximum Subnormal Value	0x0.fffffffffffffp-1022
Double.MIN_VALUE	0x0.0000000000001p-1022

API Note:

This method corresponds to the convertToHexCharacter operation defined in IEEE 754.

Parameters:

`d` - the `double` to be converted.

Returns:

a hex string representation of the argument.

Since:

1.5

valueOf

```
public static Double valueOf(String s)
                      throws NumberFormatException
```

Returns a `Double` object holding the `double` value represented by the argument string `s`.

If `s` is `null`, then a `NullPointerException` is thrown.

Leading and trailing whitespace characters in `s` are ignored. Whitespace is removed as if by the `String.trim()` method; that is, both ASCII space and control characters are removed. The rest of `s` should constitute a *FloatValue* as described by the lexical syntax rules:

FloatValue:
- $Sign_{opt}$ `NaN`
- $Sign_{opt}$ `Infinity`
- $Sign_{opt}$ *FloatingPointLiteral*
- $Sign_{opt}$ *HexFloatingPointLiteral*
- *SignedInteger*

HexFloatingPointLiteral:
- *HexSignificand BinaryExponent* $FloatTypeSuffix_{opt}$

HexSignificand:
- *HexNumeral*
- *HexNumeral* `.`
- `0x` $HexDigits_{opt}$ `.` *HexDigits*
- `0X` $HexDigits_{opt}$ `.` *HexDigits*

BinaryExponent:
- *BinaryExponentIndicator SignedInteger*

BinaryExponentIndicator:
- `p`
- `P`

where *Sign*, *FloatingPointLiteral*, *HexNumeral*, *HexDigits*, *SignedInteger* and *FloatTypeSuffix* are as defined in the lexical structure sections of *The Java Language Specification*, except that underscores are not accepted between digits. If `s` does not have the form of a *FloatValue*, then a `NumberFormatException` is thrown. Otherwise, `s` is regarded as representing an exact decimal value in the usual "computerized scientific notation" or as an exact hexadecimal value; this exact numerical value is then conceptually converted to an "infinitely precise" binary value that is then rounded to type `double` by the usual round-to-nearest rule of IEEE 754 floating-point arithmetic, which includes preserving the sign of a zero value. Note that the round-to-nearest rule also implies overflow and underflow behaviour; if the

exact value of `s` is large enough in magnitude (greater than or equal to (`MAX_VALUE` + `ulp(MAX_VALUE)`/2), rounding to `double` will result in an infinity and if the exact value of `s` is small enough in magnitude (less than or equal to `MIN_VALUE`/2), rounding to float will result in a zero. Finally, after rounding a `Double` object representing this `double` value is returned.

Note that trailing format specifiers, specifiers that determine the type of a floating-point literal (`1.0f` is a `float` value; `1.0d` is a `double` value), do *not* influence the results of this method. In other words, the numerical value of the input string is converted directly to the target floating-point type. The two-step sequence of conversions, string to `float` followed by `float` to `double`, is *not* equivalent to converting a string directly to `double`. For example, the `float` literal `0.1f` is equal to the `double` value `0.10000000149011612`; the `float` literal `0.1f` represents a different numerical value than the `double` literal `0.1`. (The numerical value 0.1 cannot be exactly represented in a binary floating-point number.)

To avoid calling this method on an invalid string and having a `NumberFormatException` be thrown, the regular expression below can be used to screen the input string:

```
final String Digits     = "(\\p{Digit}+)";
final String HexDigits  = "(\\p{XDigit}+)";
// an exponent is 'e' or 'E' followed by an optionally
// signed decimal integer.
final String Exp        = "[eE][+-]?"+Digits;
final String fpRegex    =
    ("[\\x00-\\x20]*"+  // Optional leading "whitespace"
     "[+-]?(" + // Optional sign character
     "NaN|" +           // "NaN" string
     "Infinity|" +      // "Infinity" string

     // A decimal floating-point string representing a finite positive
     // number without a leading sign has at most five basic pieces:
     // Digits . Digits ExponentPart FloatTypeSuffix
     //
     // Since this method allows integer-only strings as input
     // in addition to strings of floating-point literals, the
     // two sub-patterns below are simplifications of the grammar
     // productions from section 3.10.2 of
     // The Java Language Specification.

     // Digits ._opt Digits_opt ExponentPart_opt FloatTypeSuffix_opt
     "((("+Digits+"(\\.)?("+Digits+"?)("+Exp+")?)|"+
```

```
     // . Digits ExponentPart_opt FloatTypeSuffix_opt
     "(\\.("+Digits+")("+Exp+")?)|"+

     // Hexadecimal strings
     "((" +
      // 0[xX] HexDigits ._opt BinaryExponent FloatTypeSuffix_opt
      "(0[xX]" + HexDigits + "(\\.)?)|" +

      // 0[xX] HexDigits_opt . HexDigits BinaryExponent FloatTypeSuffix_opt
      "(0[xX]" + HexDigits + "?(\\.)" + HexDigits + ")" +

      ")[pP][+-]?" + Digits + "))" +
     "[fFdD]?))" +
     "[\\x00-\\x20]*");// Optional trailing "whitespace"
if (Pattern.matches(fpRegex, myString))
    Double.valueOf(myString); // Will not throw NumberFormatException
else {
    // Perform suitable alternative action
}
```

API Note:

To interpret localized string representations of a floating-point value, or string representations that have non-ASCII digits, use `NumberFormat`. For example,

```
    NumberFormat.getInstance(l).parse(s).doubleValue();
```

where `l` is the desired locale, or `Locale.ROOT` if locale insensitive., This method corresponds to the convertFromDecimalCharacter and convertFromHexCharacter operations defined in IEEE 754.

Parameters:

`s` - the string to be parsed.

Returns:

a `Double` object holding the value represented by the `String` argument.

Throws:

`NumberFormatException` - if the string does not contain a parsable number.

See Also:

Decimal ↔ Binary Conversion Issues

valueOf

```
public static Double valueOf(double d)
```

Returns a `Double` instance representing the specified `double` value. If a new `Double` instance is not required, this method should generally be used in preference to the constructor `Double(double)`, as this method is likely to yield significantly better space and time performance by caching frequently requested values.

Parameters:

`d` - a double value.

Returns:

a `Double` instance representing `d`.

Since:

1.5

parseDouble

```
public static double parseDouble(String s)
                          throws NumberFormatException
```

Returns a new `double` initialized to the value represented by the specified `String`, as performed by the `valueOf` method of class `Double`.

Parameters:

`s` - the string to be parsed.

Returns:

the `double` value represented by the string argument.

Throws:

`NullPointerException` - if the string is null

`NumberFormatException` - if the string does not contain a parsable `double`.

Since:

1.2

See Also:

`valueOf(String)`,
Decimal ↔ Binary Conversion Issues

isNaN

```
public static boolean isNaN(double v)
```

Returns `true` if the specified number is a Not-a-Number (NaN) value, `false` otherwise.

API Note:

This method corresponds to the isNaN operation defined in IEEE 754.

Parameters:

`v` - the value to be tested.

Returns:

`true` if the value of the argument is NaN; `false` otherwise.

isInfinite

```
public static boolean isInfinite(double v)
```

Returns `true` if the specified number is infinitely large in magnitude, `false` otherwise.

API Note:

This method corresponds to the isInfinite operation defined in IEEE 754.

Parameters:

`v` - the value to be tested.

Returns:

`true` if the value of the argument is positive infinity or negative infinity; `false` otherwise.

isFinite

```
public static boolean isFinite(double d)
```

Returns `true` if the argument is a finite floating-point value; returns `false` otherwise (for NaN and infinity arguments).

API Note:

This method corresponds to the isFinite operation defined in IEEE 754.

Parameters:

`d` - the `double` value to be tested

Returns:

`true` if the argument is a finite floating-point value, `false` otherwise.

Since:

1.8

isNaN

```
public boolean isNaN()
```

Returns `true` if this `Double` value is a Not-a-Number (NaN), `false` otherwise.

Returns:

`true` if the value represented by this object is NaN; `false` otherwise.

isInfinite

```
public boolean isInfinite()
```

Returns `true` if this `Double` value is infinitely large in magnitude, `false` otherwise.

Returns:

`true` if the value represented by this object is positive infinity or negative infinity; `false` otherwise.

toString

```
public String toString()
```

Returns a string representation of this `Double` object. The primitive `double` value represented by this object is converted to a string exactly as if by the method `toString` of one argument.

Overrides:

`toString` in class `Object`

Returns:

a `String` representation of this object.

See Also:

`toString(double)`

byteValue

```
public byte byteValue()
```

Returns the value of this `Double` as a `byte` after a narrowing primitive conversion.

Overrides:

`byteValue` in class `Number`

Returns:

the `double` value represented by this object converted to type `byte`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

Since:

1.1

shortValue

```
public short shortValue()
```

Returns the value of this `Double` as a `short` after a narrowing primitive conversion.

Overrides:

`shortValue` in class `Number`

Returns:

the `double` value represented by this object converted to type `short`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

Since:

1.1

intValue

```
public int intValue()
```

Returns the value of this `Double` as an `int` after a narrowing primitive conversion.

Specified by:

`intValue` in class `Number`

API Note:

This method corresponds to the convertToIntegerTowardZero operation defined in IEEE 754.

Returns:

the `double` value represented by this object converted to type `int`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

longValue

```
public long longValue()
```

Returns the value of this `Double` as a `long` after a narrowing primitive conversion.

Specified by:

`longValue` in class `Number`

API Note:

This method corresponds to the convertToIntegerTowardZero operation defined in IEEE 754.

Returns:

the `double` value represented by this object converted to type `long`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

floatValue

```
public float floatValue()
```

Returns the value of this `Double` as a `float` after a narrowing primitive conversion.

Specified by:

`floatValue` in class `Number`

API Note:

This method corresponds to the convertFormat operation defined in IEEE 754.

Returns:

the `double` value represented by this object converted to type `float`

See *Java Language Specification*:

5.1.3 Narrowing Primitive Conversion

Since:

1.0

doubleValue

```
public double doubleValue()
```

Returns the `double` value of this `Double` object.

Specified by:

`doubleValue` in class `Number`

Returns:

the `double` value represented by this object

hashCode

```
public int hashCode()
```

Returns a hash code for this `Double` object. The result is the exclusive OR of the two halves of the `long` integer bit representation, exactly as produced by the method `doubleToLongBits(double)`, of the primitive `double` value

represented by this Double object. That is, the hash code is the value of the expression:

```
(int)(v^(v>>>32))
```

where v is defined by:

```
long v = Double.doubleToLongBits(this.doubleValue());
```

Overrides:

hashCode in class Object

Returns:

a hash code value for this object.

See Also:

Object.equals(java.lang.Object), System.identityHashCode(Object)

hashCode

```
public static int hashCode(double value)
```

Returns a hash code for a double value; compatible with Double.hashCode().

Parameters:

value - the value to hash

Returns:

a hash code value for a double value.

Since:

1.8

equals

```
public boolean equals(Object obj)
```

Compares this object against the specified object. The result is true if and only if the argument is not null and is a Double object that represents a double that has the same value as the double represented by this object. For this purpose, two double values are considered to be the same if and only if the method doubleToLongBits(double) returns the identical long value when applied to each. In other words, representation equivalence is used to compare the double values.

Overrides:

equals in class Object

API Note:

This method is defined in terms of `doubleToLongBits(double)` rather than the == operator on `double` values since the == operator does *not* define an equivalence relation and to satisfy the equals contract an equivalence relation must be implemented; see this discussion for details of floating-point equality and equivalence.

Parameters:

`obj` - the reference object with which to compare.

Returns:

`true` if this object is the same as the obj argument; `false` otherwise.

See *Java Language Specification*:

15.21.1 Numerical Equality Operators == and !=

See Also:

`doubleToLongBits(double)`

doubleToLongBits

```
public static long doubleToLongBits(double value)
```

Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "double format" bit layout.

Bit 63 (the bit that is selected by the mask `0x8000000000000000L`) represents the sign of the floating-point number. Bits 62-52 (the bits that are selected by the mask `0x7ff0000000000000L`) represent the exponent. Bits 51-0 (the bits that are selected by the mask `0x000fffffffffffffL`) represent the significand (sometimes called the mantissa) of the floating-point number.

If the argument is positive infinity, the result is `0x7ff0000000000000L`.

If the argument is negative infinity, the result is `0xfff0000000000000L`.

If the argument is NaN, the result is `0x7ff8000000000000L`.

In all cases, the result is a `long` integer that, when given to the `longBitsToDouble(long)` method, will produce a floating-point value the same as the argument to `doubleToLongBits` (except all NaN values are collapsed to a single "canonical" NaN value).

Parameters:

`value` - a `double` precision floating-point number.

Returns:

the bits that represent the floating-point number.

doubleToRawLongBits

```
public static long doubleToRawLongBits(double value)
```

Returns a representation of the specified floating-point value according to the IEEE 754 floating-point "double format" bit layout, preserving Not-a-Number (NaN) values.

Bit 63 (the bit that is selected by the mask `0x8000000000000000L`) represents the sign of the floating-point number. Bits 62-52 (the bits that are selected by the mask `0x7ff0000000000000L`) represent the exponent. Bits 51-0 (the bits that are selected by the mask `0x000fffffffffffffL`) represent the significand (sometimes called the mantissa) of the floating-point number.

If the argument is positive infinity, the result is `0x7ff0000000000000L`.

If the argument is negative infinity, the result is `0xfff0000000000000L`.

If the argument is NaN, the result is the `long` integer representing the actual NaN value. Unlike the `doubleToLongBits` method, `doubleToRawLongBits` does not collapse all the bit patterns encoding a NaN to a single "canonical" NaN value.

In all cases, the result is a `long` integer that, when given to the `longBitsToDouble(long)` method, will produce a floating-point value the same as the argument to `doubleToRawLongBits`.

Parameters:

`value` - a `double` precision floating-point number.

Returns:

the bits that represent the floating-point number.

Since:

1.3

longBitsToDouble

```
public static double longBitsToDouble(long bits)
```

Returns the `double` value corresponding to a given bit representation. The argument is considered to be a representation of a floating-point value according to the IEEE 754 floating-point "double format" bit layout.

If the argument is `0x7ff0000000000000L`, the result is positive infinity.

If the argument is `0xfff0000000000000L`, the result is negative infinity.

If the argument is any value in the range `0x7ff0000000000001L` through `0x7fffffffffffffffL` or in the range `0xfff0000000000001L` through `0xffffffffffffffffL`, the result is a NaN. No IEEE 754 floating-point operation provided by Java can distinguish between two NaN values of the same type with different bit patterns. Distinct values of NaN are only distinguishable by use of the `Double.doubleToRawLongBits` method.

In all other cases, let *s*, *e*, and *m* be three values that can be computed from the argument:

```
int s = ((bits >> 63) == 0) ? 1 : -1;
int e = (int)((bits >> 52) & 0x7ffL);
long m = (e == 0) ?
                (bits & 0xfffffffffffffL) << 1 :
                (bits & 0xfffffffffffffL) | 0x10000000000000L;
```

Then
the floating-point result equals the value of the mathematical expression $s \cdot m \cdot 2^{e-1075}$.

Note that this method may not be able to return a `double` NaN with exactly same bit pattern as the `long` argument. IEEE 754 distinguishes between two kinds of NaNs, quiet NaNs and *signaling NaNs*. The differences between the two kinds of NaN are generally not visible in Java. Arithmetic operations on signaling NaNs turn them into quiet NaNs with a different, but often similar, bit pattern. However, on some processors merely copying a signaling NaN also performs that conversion. In particular, copying a signaling NaN to return it to the calling method may perform this conversion. So `longBitsToDouble` may not be able to return a `double` with a signaling NaN bit pattern. Consequently, for some `long` values, `doubleToRawLongBits(longBitsToDouble(start))` may *not* equal `start`. Moreover, which particular bit patterns represent signaling NaNs is platform dependent; although all NaN bit patterns, quiet or signaling, must be in the NaN range identified above.

Parameters:

`bits` - any `long` integer.

Returns:

the `double` floating-point value with the same bit pattern.

compareTo

```
public int compareTo(Double anotherDouble)
```

Compares two `Double` objects numerically. This method imposes a total order on `Double` objects with two differences compared to the incomplete order defined by the Java language numerical comparison operators (`<`, `<=`, `==`, `>=`, `>`) on `double` values.

- A NaN is *unordered* with respect to other values and unequal to itself under the comparison operators. This method chooses to define `Double.NaN` to be equal to itself and greater than all other `double` values (including `Double.POSITIVE_INFINITY`).
- Positive zero and negative zero compare equal numerically, but are distinct and distinguishable values. This method chooses to define positive zero (`+0.0d`), to be greater than negative zero (`-0.0d`).

This ensures that the *natural ordering* of `Double` objects imposed by this method is *consistent with equals*; see this discussion for details of floating-point comparison and ordering.

Specified by:

`compareTo` in interface `Comparable<Double>`

API Note:

The inclusion of a total order idiom in the Java SE API predates the inclusion of that functionality in the IEEE 754 standard. The ordering of the totalOrder predicate chosen by IEEE 754 differs from the total order chosen by this method. While this method treats all NaN representations as being in the same equivalence class, the IEEE 754 total order defines an ordering based on the bit patterns of the NaN among the different NaN representations. The IEEE 754 order regards "negative" NaN representations, that is NaN representations whose sign bit is set, to be less than any finite or infinite value and less than any "positive" NaN. In addition, the IEEE order regards all positive NaN values as greater than positive infinity. See the IEEE 754 standard for full details of its total ordering.

Parameters:

`anotherDouble` - the `Double` to be compared.

Returns:

the value `0` if `anotherDouble` is numerically equal to this `Double`; a value less than `0` if this `Double` is numerically less than `anotherDouble`; and a value greater than `0` if this `Double` is numerically greater than `anotherDouble`.

See *Java Language Specification*:

15.20.1 Numerical Comparison Operators <, <=, >, and >=

Since:

1.2

compare

```
public static int compare(double d1,
                          double d2)
```

Compares the two specified `double` values. The sign of the integer value returned is the same as that of the integer that would be returned by the call:

```
Double.valueOf(d1).compareTo(Double.valueOf(d2))
```

API Note:

One idiom to implement representation equivalence on `double` values is

```
Double.compare(a, b) == 0
```

Parameters:

`d1` - the first `double` to compare

`d2` - the second `double` to compare

Returns:

the value `0` if `d1` is numerically equal to `d2`; a value less than `0` if `d1` is numerically less than `d2`; and a value greater than `0` if `d1` is numerically greater than `d2`.

Since:

1.4

sum

```
public static double sum(double a,
                         double b)
```

Adds two `double` values together as per the + operator.

API Note:

This method corresponds to the addition operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the sum of `a` and `b`

See *Java Language Specification*:

4.2.4 Floating-Point Operations

Since:

1.8

See Also:

BinaryOperator

max

```
public static double max(double a,
                         double b)
```

Returns the greater of two `double` values as if by calling `Math.max`.

API Note:

This method corresponds to the maximum operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the greater of `a` and `b`

Since:

1.8

See Also:

BinaryOperator

min

```
public static double min(double a,
                         double b)
```

Returns the smaller of two `double` values as if by calling `Math.min`.

API Note:

This method corresponds to the minimum operation defined in IEEE 754.

Parameters:

`a` - the first operand

`b` - the second operand

Returns:

the smaller of `a` and `b`.

Since:

1.8

See Also:

BinaryOperator

describeConstable

```
public Optional<Double> describeConstable()
```

Returns an `Optional` containing the nominal descriptor for this instance, which is the instance itself.

Specified by:

`describeConstable` in interface `Constable`

Returns:

an `Optional` describing the Double instance

Since:

12

resolveConstantDesc

```
public Double resolveConstantDesc(MethodHandles.Lookup lookup)
```

Resolves this instance as a `ConstantDesc`, the result of which is the instance itself.

Specified by:

`resolveConstantDesc` in interface `ConstantDesc`

Parameters:

`lookup` - ignored

Returns:

the Double instance

Since:

12

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.